Cuckoo Hashing

CS648 Course Project Report

by Madhav lata, Nikhil Jain and Sambuddha Chakrabarti

Under the supervision of Dr. Surendra baswana

April 2026

Group Members

Name	Roll Number	Email ID
Madhav Lata	220597	madhavlata22@iitk.ac.in
Nikhil Jain	220709	nikhilj22@iitk.ac.in
Sambuddha Chakrabarti	220947	sambuddha22@iitk.ac.in

Contents

1 Introduction **2**
- 1.1 Motivation . . . 2
- 1.2 Historical Context . . . 2
- 1.3 Scope of This Report . . . 2

2 The Cuckoo Hashing Mechanism **2**
- 2.1 Data Structure . . . 2
- 2.2 Insertion: The Cuckoo Displacement . . . 3
- 2.3 Rehashing . . . 3

3 Mathematical Analysis and Proofs **4**
- 3.1 Setup and Notation . . . 4
- 3.2 Hash Function Families . . . 4
- 3.3 Lookup: Worst-Case $O(1)$ Time . . . 5
- 3.4 Insertion Procedure . . . 5
- 3.5 Analysis of the Insertion Procedure . . . 6
- 3.6 Cost of Rehashing . . . 8
- 3.7 Summary of Complexity . . . 8

4 Advanced Variants of Cuckoo Hashing **8**
- 4.1 Asymmetric Cuckoo Hashing . . . 9
- 4.2 Single-Table Variant . . . 10

5 Comparative Analysis: Alternative Algorithms **11**
- 5.1 Open Addressing Schemes . . . 11
  - 5.1.1 Linear Probing . . . 11
  - 5.1.2 Double Hashing . . . 11
- 5.2 Chaining Methods . . . 12
  - 5.2.1 Chained Hashing . . . 12
  - 5.2.2 Two-Way Chaining . . . 12
- 5.3 Overall Comparison . . . 12

6 Experiments **13**
- 6.1 Experimental Setup . . . 13
- 6.2 Experiment 1 — The Probe Explosion . . . 14
- 6.3 Experiment 2 — The Hardware Illusion . . . 15
- 6.4 Experiment 3 — Lookup Latency: All Schemes . . . 17
- 6.5 Experiment 4 — Symmetric vs. Asymmetric Cuckoo Hashing . . . 18
- 6.6 Summary of Experimental Findings . . . 19

1 Introduction

The dictionary data structure is a fundamental component in computer science, used to maintain a set S under insertion, deletion, and membership queries. While traditional hashing techniques offer excellent average-case performance, they lack robust worst-case guarantees — particularly for lookup time, which is the most critical operation in many applications. **Cuckoo Hashing**, introduced by Rasmus Pagh and Flemming Friche Rodler in 2001, addresses this gap by providing **worst-case constant lookup time** with a structure simple enough for practical use.

1.1 Motivation

The primary motivation is the need for reliability in time-critical systems. Applications such as **high-speed IP routing**, **real-time systems**, and **hardware-level caches** cannot tolerate the $O(\log n)$ or $O(n)$ lookup spikes that arise from collisions in traditional schemes. A lookup that always terminates in exactly two memory accesses — regardless of table occupancy — eliminates this uncertainty entirely.

1.2 Historical Context

Achieving worst-case constant lookup previously required complex perfect hashing schemes. The static construction of Fredman, Komlós, and Szemerédi (1984) and the dynamic scheme of Dietzfelbinger et al. (1994) both achieved $O(1)$ worst-case lookup, but at the cost of up to $35n$ words of memory and considerable implementation complexity. Cuckoo Hashing matches their theoretical guarantees using only $\approx 2n$ words and a straightforward insertion procedure. A detailed comparison with these and other schemes is presented in Section 5.

1.3 Scope of This Report

In this report we implement and analyse the standard Cuckoo Hashing algorithm. Section 2 describes the core mechanism. Section 3 develops the full mathematical analysis. Section 4 covers advanced variants. Section 5 compares Cuckoo Hashing against alternative schemes. Section 6 presents our experimental results on modern hardware.

2 The Cuckoo Hashing Mechanism

Cuckoo Hashing guarantees $O(1)$ worst-case lookup by restricting every key to **exactly two possible locations** — one in each of two hash tables. A lookup therefore never inspects more than two cells, regardless of how many keys are stored.

2.1 Data Structure

The structure maintains two hash tables T_1 and T_2, each of size $r \geq (1+\varepsilon)n$ for a fixed constant $\varepsilon > 0$, and two independent hash functions $h_1, h_2 : U \to \{0, \ldots, r-1\}$. Every key $x \in S$ is stored in *exactly one* of:

$$T_1[h_1(x)] \qquad \text{or} \qquad T_2[h_2(x)].$$

The lookup function is consequently:
This is exactly **two memory accesses** in every case. Deletion is equally simple: locate the key via lookup and clear its cell in $O(1)$ worst-case time.

Algorithm 1 LOOKUP(x)

return $T_1[h_1(x)] = x \ \vee \ T_2[h_2(x)] = x$

2.2 Insertion: The Cuckoo Displacement

Insertion follows a greedy displacement strategy. To insert key x:

1. If $T_1[h_1(x)]$ is empty, place x there and stop.
2. Otherwise, place x in $T_1[h_1(x)]$ anyway, evicting the current occupant y (making it *nestless*).
3. Attempt to place y in its alternative location in T_2. If that cell is also occupied, evict its occupant and continue.
4. Repeat until a vacant cell is found or `MaxLoop` iterations are exhausted.

Algorithm 2 INSERT(x)

```
if LOOKUP(x) then return
for MaxLoop times do
    x ↔ T1[h1(x)]
    if x = ⊥ then return
    x ↔ T2[h2(x)]
    if x = ⊥ then return
REHASH(); INSERT(x)
```

2.3 Rehashing

If the displacement chain reaches `MaxLoop` steps, the current hash functions cannot accommodate the key set and a **rehash** is triggered: new functions h_1, h_2 are chosen and all keys are re-inserted. Although a rehash costs $O(n)$, the probability of any single insertion triggering one is only $O(1/n^2)$, keeping the amortized expected insertion time $O(1)$. The full proof of this bound is given in Section 3.

Operation	**Time**	**Guarantee**
Lookup	$O(1)$	Worst case
Delete	$O(1)$	Worst case
Insert	$O(1)$	Amortized expected
Space	$2n$ words	—

3 Mathematical Analysis and Proofs

In this section, we formally analyze the performance of Cuckoo Hashing. We will prove that the lookup operation always takes $O(1)$ worst-case time, and that insertion takes $O(1)$ amortized expected time. We will build up the analysis step by step, carefully defining every term we use.

3.1 Setup and Notation

We begin by establishing the setting in which Cuckoo Hashing operates.

- Let U be the **universe** — the set of all possible keys (e.g. all 32-bit integers).
- Let $S \subseteq U$ be the **current set** of keys stored in the dictionary, with $|S| = n$.
- We maintain **two hash tables** T_1 and T_2, each of size r, where

  $$r \;\geq\; (1+\varepsilon)\, n$$

  for some fixed constant $\varepsilon > 0$. This means each table is slightly larger than n, so neither table is ever more than half full. The parameter ε controls how much "slack" we keep; a larger ε gives better performance at the cost of more memory.
- We use **two hash functions** $h_1, h_2 : U \to \{0, 1, \ldots, r-1\}$, chosen independently and at random from a suitable family (described in Section 3.2).
- Every key $x \in S$ is stored in *exactly one* of two possible cells: either $T_1[h_1(x)]$ or $T_2[h_2(x)]$.

The special symbol $\perp$ ("bottom") is used to denote an **empty cell** in either table.

3.2 Hash Function Families

What is a hash function family? A *hash function family* is simply a large collection of hash functions from which we can pick one at random. Using a randomly chosen function (rather than a single fixed function) prevents adversarial inputs from causing worst-case behaviour.

Universal hash families. The analysis requires hash functions that behave "almost like" truly random functions, but that are efficient to compute. We use the following formal definition.

Definition 3.1 (**(c,k)-Universal Family**). *A family $\{h_i\}_{i \in I}$, where each $h_i : U \to R$, is called* (c,k)-universal *if, for any k* ***distinct*** *keys $x_1, \ldots, x_k \in U$, any target values $y_1, \ldots, y_k \in R$, and a uniformly random index $i \in I$,*

$$\Pr\big[h_i(x_1) = y_1,\ \ldots,\ h_i(x_k) = y_k\big] \;\leq\; \frac{c}{|R|^k}.$$

Intuition. If $c = 1$, this says that the probability of any k keys all landing at prescribed locations is *at most* as large as it would be if the hash function were truly random. In other words, a $(1,k)$-universal family "looks random" on every set of k keys.

The hash functions used in practice. For our theoretical guarantees, we use a construction due to Siegel that yields a family which, when restricted to any set of r^2 keys, is $(1, n^\delta)$-universal for some constant $\delta > 0$, with probability $1 - O(1/n^2)$. These functions have **constant evaluation time** and require only $o(n)$ words of storage. Because we allow at most r^2 insertions before forcing a rehash (see Section 3.4), the family behaves like a truly random function on all keys we ever process in a single "epoch".

3.3 Lookup: Worst-Case $O(1)$ Time

The lookup procedure is strikingly simple:

Algorithm 3 LOOKUP(x)

```
return T_1[h_1(x)] = x ∨ T_2[h_2(x)] = x
```

Since every key $x \in S$ is stored in *exactly one* of $T_1[h_1(x)]$ or $T_2[h_2(x)]$, checking both cells is both necessary and sufficient. This is **exactly 2 memory accesses** regardless of n, giving

$$\text{Lookup time} \;=\; O(1) \quad \textbf{worst case.}$$

Moreover, the two memory accesses are *independent* of each other and can be issued in parallel on hardware that supports it.

3.4 Insertion Procedure

The "cuckoo" idea. The name comes from the cuckoo bird, which lays its egg in another bird's nest, evicting the original occupant. Our insertion does exactly this:

Algorithm 4 INSERT(x)

```
if LOOKUP(x) then return
for MaxLoop times do
    x ↔ T_1[h_1(x)]                ▷ Swap x with the current occupant of T_1[h_1(x)]
    if x = ⊥ then return           ▷ We found an empty cell; done
    x ↔ T_2[h_2(x)]                ▷ Swap x with the current occupant of T_2[h_2(x)]
    if x = ⊥ then return
REHASH(); INSERT(x)                ▷ Cycle detected; pick new hash functions and retry
```

Key terms:

- **Nestless key**: a key that has been evicted from its cell and has not yet found a new home.
- **Closed loop**: a situation where the eviction chain returns to a previously visited cell, forming a cycle. This means it is impossible to place all keys with the current hash functions.
- **Rehash**: choosing fresh hash functions h_1, h_2 and re-inserting all keys. This costs $O(n)$ expected time but happens very rarely.
- **MaxLoop**: the maximum number of swap iterations we allow before declaring failure and rehashing. We set

$$\text{MaxLoop} \;=\; \lceil 3 \log_{1+\varepsilon} r \rceil.$$

Epoch control. If the hash tables have size r, we also force a rehash after r^2 insertions, even if no insertion has failed. This ensures the theoretical guarantees on the hash functions remain valid throughout the epoch.

3.5 Analysis of the Insertion Procedure

We now prove that insertion runs in $O(1)$ **amortized expected** time. The analysis has three parts.

Part 1: Structure of the Eviction Chain

Consider the sequence of nestless keys $x_1, x_2, x_3, \ldots$ produced during a single insertion of x_1.

Case A — No cell is revisited. The sequence $x_1, x_2, \ldots$ consists of distinct keys. The procedure terminates as soon as it reaches an empty cell.

Case B — A cell is revisited (Figure 1). Suppose cell c is first visited at step i and revisited at step $j > i$. The key x_i becomes nestless for a second time and is placed back in its original cell. All keys $x_{i-1}, \ldots, x_2$ are then moved back to where they started (a "backtrack"). Key x_1 becomes nestless again and attempts to enter T_2 instead of T_1.

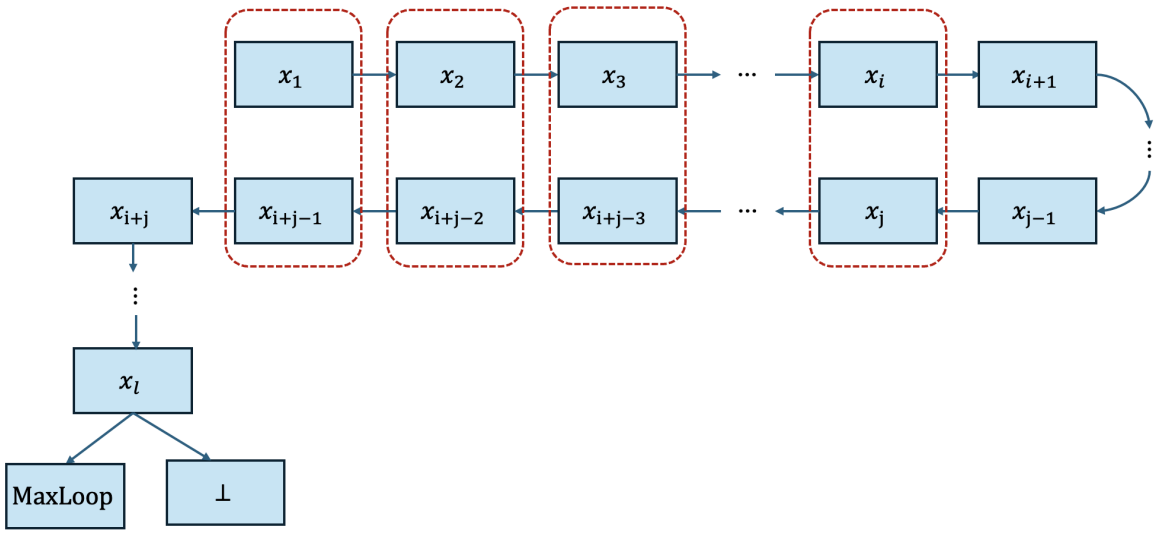

Figure 1: Three stages of an insertion of key x_1, involving the movement of keys $x_1, \ldots, x_l$. Boxes correspond to cells in either of the two tables, and arcs show possibilities for moving keys. A bold arc shows where the nestless key is to be inserted. The sequence returns to a previously visited cell at step j, forming a closed loop.

Lemma: Suppose that the insertion procedure does not enter a `MaxLoop`. Then for any prefix $x_1, x_2, \ldots, x_p$ of the sequence, there must be a subsequence of at least $p/3$ consecutive keys without repetitions, starting with occurrence of x_1.

Proof: Let $X_{u,v}$ denote the length of sequence between keys u and v.

$$\underbrace{X_{1,j-1}} + X_{j,i+j-2} + X_{i+j-1,p} = p$$

$$\xrightarrow{\text{since}} \quad \underline{X_{1,j-1} \;\geq\; X_{j,i+j-2}}$$

$$2X_{1,j-1} + X_{i+j-1,p} \;\geq\; p$$

This implies at least one of $X_{1,j-1}$ or $X_{i+j-1,p} \geq p/3$. ■

Part 2: Probability Bounds on Long Eviction Chains

We bound the probability that the insertion loop runs for at least t iterations. There are three scenarios under which this can happen (they are not mutually exclusive):

1. **Hash function failure.** The chosen family is not $(1, \text{MaxLoop})$-universal on the current key set. By construction (Siegel's theorem), this occurs with probability $O(1/n^2)$.

2. **A closed loop forms.** The eviction chain revisits a cell for a second time, creating a cycle. Let $v \leq l$ be the number of *distinct* nestless keys involved. Counting all ways the cycle can form (positions i and j, the closing cell, the choice of cells, and the choice of keys), there are at most $v^3 r^{v-1} n^{v-1}$ possible configurations. Since $v \leq \text{MaxLoop}$, the hash functions are $(1, v)$-universal, so each configuration occurs with probability at most r^{-2v}. Summing over all $v \geq 3$ and using $r/n > 1 + \varepsilon$:
$$\sum_{v=3}^{l} v^3 r^{v-1} n^{v-1} r^{-2v} \;\leq\; \frac{1}{rn} \sum_{v=3}^{\infty} v^3 (n/r)^v \;=\; O\left(\frac{1}{n^2}\right).$$

3. **A long chain without a cycle.** By Lemma **??**, iteration t requires a sequence of $v = \lceil (2t-1)/3 \rceil$ distinct nestless keys $b_1, \ldots, b_v$ satisfying the "alternating collision" pattern:
$$h_{\beta_1}(b_1) = h_{\beta_1}(b_2), \quad h_{\beta_2}(b_2) = h_{\beta_2}(b_3), \quad h_{\beta_1}(b_3) = h_{\beta_1}(b_4), \quad \ldots$$
for some choice of $(\beta_1, \beta_2) \in \{(1,2), (2,1)\}$. Given b_1, there are at most n^{v-1} ways to choose the remaining keys. The probability that all $v-1$ hash collisions hold simultaneously is at most $r^{-(v-1)}$ (by $(1, \text{MaxLoop})$-universality). Hence the probability of this scenario is at most
$$2(n/r)^{v-1} \;\leq\; 2(1+\varepsilon)^{-(2t-1)/3+1}. \tag{1}$$

Part 3: Expected Number of Iterations is $O(1)$

Combining the bounds above, the expected number of iterations in the insertion loop satisfies:

$$\begin{aligned}
\mathbb{E}[\text{iterations}] &\leq 1 + \sum_{t=2}^{\text{MaxLoop}} \left(2(1+\varepsilon)^{-(2t-1)/3+1} + O(1/n^2)\right) \\
&\leq 1 + O\left(\frac{\text{MaxLoop}}{n^2}\right) + 2\sum_{t=0}^{\infty} \left((1+\varepsilon)^{-2/3}\right)^t \\
&= O\left(1 + \frac{1}{1-(1+\varepsilon)^{-2/3}}\right) \\
&= O(1 + 1/\varepsilon).
\end{aligned} \tag{2}$$

Since ε is a fixed constant, this is $O(1)$.

3.6 Cost of Rehashing

A rehash is triggered when the insertion loop hits MaxLoop iterations. We now show that this happens very rarely.

Theorem 3.1. *The probability that any single insertion causes a rehash is $O(1/n^2)$.*

Proof. Setting MaxLoop $= \lceil 3 \log_{1+\varepsilon} r \rceil$, the probability of a rehash due to scenario 2 or 3 above is at most

$$2(1+\varepsilon)^{-(2\,\mathrm{MaxLoop}-1)/3+1} \;=\; O\left(\frac{1}{n^2}\right),$$

by (1). Adding the $O(1/n^2)$ probability from scenario 1 gives a total rehash probability of $O(1/n^2)$. ■

Cost of a rehash. A rehash re-inserts all n current keys using new hash functions. Each re-insertion succeeds (causes no further rehash) with probability $1 - O(1/n)$. Therefore the expected total time for a rehash is $O(n)$. Since a rehash is triggered by any single insertion with probability $O(1/n^2)$, the expected cost attributable to rehashing *per insertion* is:

$$O(n) \times O(1/n^2) \;=\; O(1/n) \;=\; O(1).$$

There is also a **scheduled rehash** after r^2 insertions (even with no failures). Its amortized cost per insertion is $O(n)/r^2 = O(1/n)$, which is also $O(1)$.

3.7 Summary of Complexity

Theorem 3.2 (Performance of Cuckoo Hashing). *Let $\varepsilon > 0$ be a fixed constant and suppose each table has size $r \geq (1+\varepsilon)n$. Using Siegel's hash function family, Cuckoo Hashing achieves:*

Operation	***Time Complexity***	***Type***
Lookup	$O(1)$	*Worst case*
Delete	$O(1)$	*Worst case*
Insert	$O(1)$	*Amortized expected*

The total space used is $2r = O(n)$ words.

The key insight is that lookup is always $O(1)$ in the *worst case* because every key has **at most two possible locations**, and we check both. Insertion is efficient in the *expected* sense because long eviction chains are exponentially unlikely, and rehashes are triggered with only inverse-polynomial probability.

4 Advanced Variants of Cuckoo Hashing

The basic two-table Cuckoo Hashing scheme described in the previous sections is already a significant theoretical and practical achievement. However, several natural generalizations and modifications improve either the **space utilization**, the **lookup speed**, or the **implementation simplicity**. In this section we describe three such variants in detail.

4.1 Asymmetric Cuckoo Hashing

Motivation. In the standard (symmetric) scheme, the two tables T_1 and T_2 have the same size r. The insertion procedure always tries to place a key in T_1 first, which means that — even in the symmetric scheme — slightly more keys end up residing in T_1 than in T_2. Experiments reported in the original paper confirm this: after a long sequence of mixed insertions and deletions at load factor 1/3, approximately **63%** of keys reside in T_1 in the symmetric scheme.

The idea behind *Asymmetric Cuckoo Hashing* is to deliberately **make T_1 larger than T_2**, amplifying this natural bias even further. In the variant studied experimentally, T_1 is set to be **twice the size** of T_2. That is, if the total memory budget allows for $2r$ cells, we allocate $\frac{4r}{3}$ cells to T_1 and $\frac{2r}{3}$ cells to T_2 (keeping the total the same as before).

Effect on key distribution. Because T_1 is larger, it has more empty cells at any given load factor, so the insertion procedure finds a free slot in T_1 more often. This shifts the key distribution significantly. Experimentally, after the same sequence of operations at load factor 1/3, the asymmetric scheme places approximately **76%** of keys in T_1 — a notable increase over the 63% in the symmetric case.

Why does this help lookups? Recall the lookup procedure:

Algorithm 5 LOOKUP(x) — both symmetric and asymmetric

```
if T_1[h_1(x)] = x then return true
if T_2[h_2(x)] = x then return true
return false
```

On modern hardware, we always check T_1 first. If the key is found there, the second memory access to T_2 is **avoided entirely**. Since 76% of keys reside in T_1 in the asymmetric scheme (versus 63% in the symmetric scheme), a larger fraction of successful lookups terminate after just **one** memory access. This directly reduces the average lookup time for successful queries.

Effect on other operations. The insertion time is only *slightly* worse than in the symmetric scheme, because the eviction chain dynamics are very similar. Deletion remains $O(1)$ worst case regardless of the table sizes. Therefore, the asymmetric variant offers a **free improvement** in successful lookup speed at essentially no cost to other operations.

Formal summary.

Property	**Symmetric**	**Asymmetric**
Size of T_1	r	$\frac{4r}{3}$
Size of T_2	r	$\frac{2r}{3}$
Total space	$2r$	$2r$
Lookup (worst case)	$O(1)$	$O(1)$
Insertion (amortized)	$O(1)$	$O(1)$

4.2 Single-Table Variant

Motivation. The standard scheme uses *two separate arrays* T_1 and T_2. In some settings it is more convenient — for cache locality, memory allocation, or implementation simplicity — to work with a **single contiguous array** of size $2r$. The single-table variant achieves exactly this, while preserving all theoretical guarantees.

Construction. We maintain one table T of size $2r$ and two hash functions $h_1, h_2 : U \to \{0, 1, \ldots, 2r-1\}$. Each key x has two candidate locations in T:

$$\text{Location 1}: \quad h_1(x), \qquad \text{Location 2}: \quad \big(h_2(x) - h_1(x)\big) \bmod 2r.$$

The key observation is:

> *If a key x currently sits at position i in T, we can jump to its other location using the map*
>
> $$i \longmapsto \big(h_2(x) - i\big) \bmod 2r.$$

Why does this work? Notice that the two locations of x always *sum to* $h_2(x) \pmod{2r}$:

$$h_1(x) + \big(h_2(x) - h_1(x)\big) \equiv h_2(x) \pmod{2r}.$$

Therefore, given the current position i of a nestless key x, its other position is simply $h_2(x) - i \pmod{2r}$. This means we **do not need to store which hash function placed the key** — we can always compute the alternate location on the fly. The insertion loop becomes:

Algorithm 6 INSERT(x) — Single-Table Variant

```
if LOOKUP(x) then return
pos ← h1(x)
for MaxLoop times do
    x ↔ T[pos]                          ▷ Evict current occupant
    if x = ⊥ then return
    pos ← (h2(x) − pos) mod 2r          ▷ Jump to the other location
REHASH(); INSERT(x)
```

Advantages over the two-table scheme.

- **Single memory allocation**: only one array of size $2r$ is needed, which may be friendlier to the memory allocator and to hardware prefetching.
- **No bookkeeping**: we never need to record which of the two hash functions placed a given key, since the alternate location is always recoverable via the XOR-like map above.
- **Same asymptotic guarantees**: the analysis of Section 3 carries over essentially unchanged, since the probabilistic structure of the two locations is the same as in the two-table scheme.

Lookup in the single-table variant. This is still exactly **two memory accesses** in the worst case, preserving the $O(1)$ worst-case lookup guarantee.

Algorithm 7 LOOKUP(x) — Single-Table Variant

1: $\ell_1 \leftarrow h_1(x)$
2: $\ell_2 \leftarrow (h_2(x) - h_1(x)) \bmod 2r$ **return** $T[\ell_1] = x \ \vee \ T[\ell_2] = x$

Comparison of both variants.

Variant	Tables	Lookup	Insert (amortized)	Max utilization
Standard (symmetric)	2	$O(1)$	$O(1)$	$\approx 50\%$
Asymmetric	2	$O(1)$	$O(1)$	$\approx 50\%$
Single-table	1	$O(1)$	$O(1)$	$\approx 50\%$

The asymmetric and single-table variants are primarily **engineering improvements** — they keep the same asymptotic guarantees while improving practical constants or simplifying implementation.

5 Comparative Analysis: Alternative Algorithms

We compare Cuckoo Hashing against the most prominent dictionary data structures. Throughout, n denotes the number of stored keys and $\alpha \in (0, 1)$ the **load factor** — the fraction of occupied cells.

5.1 Open Addressing Schemes

In *open addressing*, all keys are stored directly inside a single array. On a collision, the new key is placed in an alternative cell determined by a **probe sequence**. There are no pointers or auxiliary structures.

5.1.1 Linear Probing

Core idea. Key x is stored in the first empty cell of the sequence $h(x),\ h(x) + 1,\ h(x) + 2,\ \ldots \pmod r$.

Complexity. Expected probes per insertion at load factor α:

$$\mathbb{E}[\text{probes}] \ = \ \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right).$$

The expected *longest* probe sequence is $\Omega(\log n)$, so there is **no worst-case guarantee**. Its main advantage is **excellent cache behaviour** — consecutive probes hit the same cache line — but this advantage is architecture-dependent and diminishes for large dictionaries. It also suffers from **primary clustering** at high load factors.

5.1.2 Double Hashing

Core idea. Two hash functions h and g are used. The probe sequence is $h(x),\ h(x)+g(x),\ h(x)+2g(x),\ \ldots \pmod r$, where the step size $g(x)$ is key-dependent, spreading probes more uniformly and avoiding primary clustering.

Complexity.

$$\mathbb{E}[\text{probes}] \;=\; \frac{1}{1-\alpha}.$$

Strictly better than Linear Probing at the same α, but the worst-case probe length is still $\Omega(\log n)$. Deletions require **"deleted" markers**, which accumulate and degrade performance over time.

5.2 Chaining Methods

In *separate chaining*, each table cell holds a pointer to a linked list of all keys mapping to that cell. Collisions are resolved by appending to the list.

5.2.1 Chained Hashing

Core idea. A table of size n/α is maintained. Cell i holds all keys x with $h(x) = i$ in a linked list.

Complexity. Under a universal hash family, all operations run in $O(1)$ expected time. However the **longest chain** — the worst-case lookup cost — grows as:

$$\mathbb{E}[\text{max chain length}] \;=\; \Theta\left(\frac{\log n}{\log\log n}\right).$$

Pointer-based storage also increases memory usage and causes frequent cache misses.

5.2.2 Two-Way Chaining

Core idea. Two independent hash functions h_1, h_2 and two tables are used. Each key is inserted into whichever of the two candidate chains is currently shorter — the **power of two choices**.

Complexity. This simple change reduces the expected maximum chain length dramatically:

$$\mathbb{E}[\text{max chain length}] \;=\; O(\log\log n).$$

Both Two-Way Chaining and Cuckoo Hashing exploit two candidate locations per key. The difference is that Two-Way Chaining *appends* to a chain (worst-case $O(\log\log n)$), while Cuckoo Hashing enforces *at most one key per cell* (worst-case $O(1)$). Two-Way Chaining is also limited to much lower maximum load factors.

5.3 Overall Comparison

Scheme	**Lookup (avg)**	**Insert (amort.)**	**Space**	**Worst-case lookup**
Linear Probing	$O(1)$	$O(1)$	n words	No $(\Omega(\log n))$
Double Hashing	$O(1)$	$O(1)$	n words	No $(\Omega(\log n))$
Chained Hashing	$O(1)$	$O(1)$	$> n$ words	No $(\Theta(\frac{\log n}{\log\log n}))$
Two-Way Chaining	$O(1)$	$O(1)$	$O(n)$ words	No $(O(\log\log n))$
Cuckoo Hashing	$O(1)$	$O(1)$	$2n$ words	**Yes** $(O(1))$

Cuckoo Hashing is the only scheme that simultaneously achieves $O(1)$ **worst-case** lookup, $O(1)$ amortised updates, and linear space with a small constant — making it both theoretically optimal and practically competitive.

6 Experiments

In this section we present the experimental evaluation of Cuckoo Hashing against five competing dictionary architectures. Our goal is to empirically validate the theoretical claims of Pagh and Rodler on modern hardware, and to expose the interplay between algorithmic complexity and physical memory architecture — a dimension the original 2001 paper could only partially explore given the hardware available at the time.

6.1 Experimental Setup

Hardware and environment. All experiments were conducted on an **Intel Core i7-10750H** processor and **12MB L3 cache**. All implementations were written from scratch in **C++23** with full compiler optimisations enabled. Timing uses `std::chrono::high_resolution_clock`. The entire benchmark suite ran uninterrupted for over **26 hours**, accumulating approximately **3 trillion memory operations** in total.

Algorithms implemented. We implement and benchmark six hash table architectures, all sharing a common interface:

Algorithm	**Key implementation detail**
Linear Probing (Standard)	Contiguous 8-byte slots. Sequential probing exploits CPU spatial prefetching — multiple slots per 64-byte cache line.
Linear Probing (Padded)	Each slot padded to exactly **64 bytes** via `alignas(64)`, forcing one cache miss per probe. Isolates pure algorithmic cost from hardware assistance.
Double Hashing	Step size $g(x) = (h_2(x) \bmod (r-1)) + 1$. Eliminates primary clustering.
Two-Way Chaining	Key inserted into the shorter of two candidate chains. Probe count = 2 + (chain length traversed).
Cuckoo Symmetric (1:1)	Two tables each of size $r/2$.
Cuckoo Asymmetric (2:1)	T_1 has $2r/3$ slots, T_2 has $r/3$ slots. Same total memory as Symmetric.

Hash functions. For open-addressing schemes we use a multiplicative family:

$$h_a(x) \;=\; (a \cdot x) \;\oplus\; \big((a \cdot x) \gg 16\big), \quad a \text{ odd.}$$

For Cuckoo Hashing — which is known to be sensitive to hash function quality — we use a stronger three-way XOR composition:

$$H(x) \;=\; h_{a_1}(x) \;\oplus\; h_{a_2}(x) \;\oplus\; h_{a_3}(x),$$

with three independently resampled odd multipliers per trial.

Benchmarking methodology. A fixed dataset of $\approx 10^6$ distinct random 32-bit keys is pre-generated once. For each (algorithm, load factor) pair we run **5,000 independent trials**. In each trial: (1) a fresh table is allocated with freshly resampled hash functions; (2) exactly $\lfloor \alpha \cdot 2^{20} \rfloor$ keys are inserted and timed; (3) the same keys are looked up and timed. Per-trial averages are written to a raw CSV file. Trials that fail are individually skipped and not counted toward the averages, so single unlucky hash function draws do not corrupt neighbouring load factor points.

Parameter	**Value**
Hardware	Intel i7-10750H
L3 Cache	12 MB
Language / Standard	C++23, full optimisations
Table size	2^{20} slots ($\approx$ 1 million)
Trials per point	5,000
Total operations	$\approx$ 3 trillion
Total runtime	$>$ 26 hours
Load factor range (open)	$\alpha \in [0.02,\ 0.98]$, step 0.02
Load factor range (Cuckoo)	$\alpha \in [0.02,\ 0.49]$, step 0.02

Table 1: Summary of experimental parameters.

6.2 Experiment 1 — The Probe Explosion

Claim being tested. The paper claims Cuckoo Hashing achieves $O(1)$ amortized expected insertion cost (measured in displacement kicks) at any load factor $\alpha < 0.5$. Open addressing schemes, by contrast, have expected insertion probe counts of $\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$ for Linear Probing and $\frac{1}{1-\alpha}$ for Double Hashing — both growing without bound as $\alpha \to 1$.

What we measure. The `insert` method returns the number of **probes or displacement kicks** taken. For open addressing, this is the number of cells examined before finding an empty slot (minimum 1). For Two-Way Chaining it is 2 header checks plus the length of the chain traversed. For Cuckoo Hashing it is the number of eviction kicks before the key settles. We plot the average probes/kicks per insertion against α on a log-log scale to capture the full range of behaviour.

Results.

Discussion. Figure 2 demonstrates the probe explosion vividly. Several points are worth noting:

- **Linear Probing** diverges most aggressively, exceeding **10 probes per insertion** at $\alpha \approx 0.95$ due to primary clustering. The $\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$ formula predicts this shape precisely.
- **Double Hashing** grows more gracefully following $1/(1-\alpha)$, as step-size randomisation eliminates primary clustering.

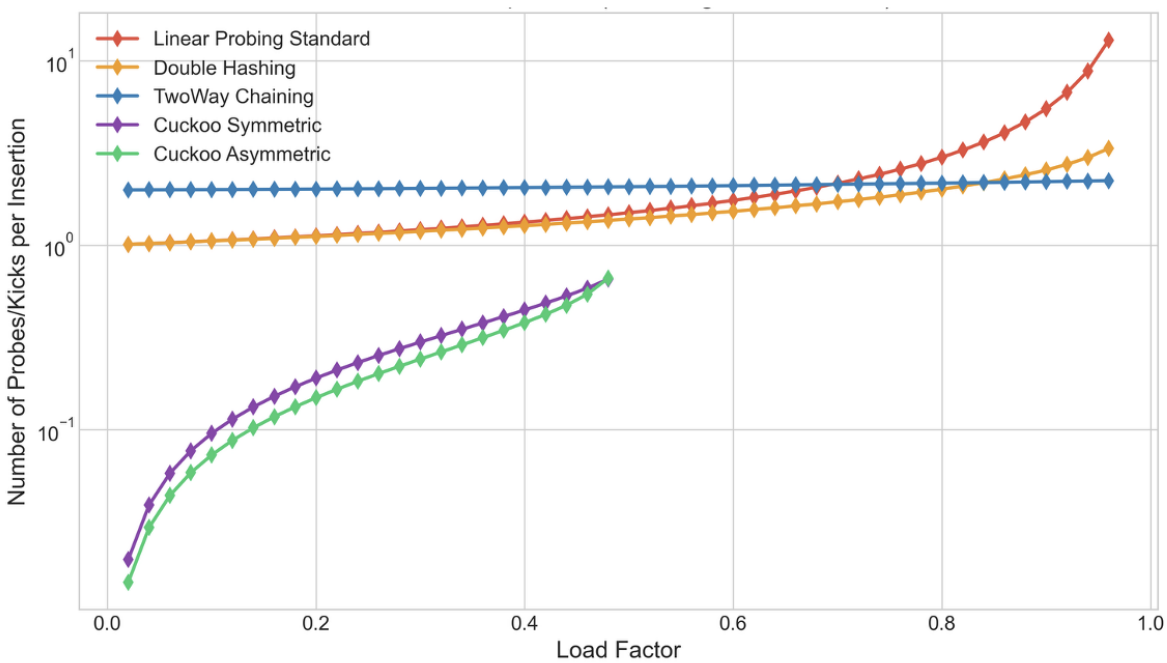

Figure 2: **The Probe Explosion.** Average probes or kicks per insertion on a log-log scale. Linear Probing and Double Hashing diverge exponentially beyond $\alpha = 0.7$. Two-Way Chaining remains flat near 2 probes due to the power of two choices. Both Cuckoo variants stay well below 1 kick per insertion throughout their valid range, with the Asymmetric variant consistently requiring fewer kicks than the Symmetric variant.

- **Two-Way Chaining** is nearly flat at ≈ 2 probes across the entire range, confirming the $O(\log \log n)$ worst-case chain length guarantee. The flat line reflects that the "power of two choices" keeps chain lengths near-constant regardless of load.
- **Cuckoo Hashing (both variants)** operates well below 1 kick per insertion on average, validating the $O(1 + 1/\varepsilon)$ amortized bound. The **Asymmetric (2:1)** variant consistently requires fewer kicks than Symmetric because the larger T_1 presents more empty cells at each step of the eviction chain.
- Both Cuckoo variants hit a **hard ceiling at** $\alpha = 0.49$, beyond which cyclical bipartite loops in the Cuckoo graph become unavoidable. This is consistent with the theoretical $\alpha < 0.5$ per-table limit.

6.3 Experiment 2 — The Hardware Illusion

Claim being tested. Pagh and Rodler note that *"the excellent cache usage of Linear Probing makes it a prime choice on modern architectures."* We isolate exactly how much of Linear Probing's speed comes from the algorithm itself versus from the CPU's hardware prefetcher. We call this the **Hardware Illusion**: the apparent speed of Linear Probing at low load is a property of the memory architecture, not of the algorithm.

α	**Lin. Prob.**	**Dbl. Hash**	**Two-Way**	**Cuckoo Sym.**	**Cuckoo Asym.**
0.10	≈ 1.1	≈ 1.1	≈ 2.0	≈ 0.09	≈ 0.08
0.30	≈ 1.5	≈ 1.4	≈ 2.0	≈ 0.25	≈ 0.20
0.49	≈ 2.5	≈ 2.0	≈ 2.1	≈ 0.60	≈ 0.55
0.70	≈ 6.0	≈ 3.3	≈ 2.1	—	—
0.95	$\approx 12+$	≈ 4.0	≈ 2.2	—	—

Table 2: Average probes/kicks per insertion at selected load factors. Cuckoo Hashing is valid only up to $\alpha < 0.5$.

What we measure. We compare **average lookup latency in nanoseconds per key** across two carefully designed variants of Linear Probing:

- **Standard (8-byte slots)**: consecutive probes access consecutive memory addresses. A single 64-byte cache line fetch brings **8 slots** into the CPU cache simultaneously. Multiple probes are served from the same cache line at near-zero marginal cost.
- **Padded (64-byte slots, `alignas(64)`)**: each slot occupies exactly one cache line. Every probe requires a **separate cache miss**. This is the true algorithmic cost of linear probing, stripped of all hardware assistance.

Cuckoo Hashing (Symmetric) is included as a baseline. Its two memory accesses are always to random locations, so it never benefits from spatial locality.

Results.

Discussion. Figure 3 exposes the hardware illusion clearly:

- **At very low load** ($\alpha < 0.1$), the Standard and Padded variants have nearly identical latency ($\approx$ 27–28 ns). At this density, average probe sequences are short enough that the cache advantage of standard layout is minimal.
- **Beyond** $\alpha \approx 0.15$, the Padded variant begins to diverge upward. Each additional probe now costs a full L3 cache miss ($\approx$ 30–40 ns on our hardware). By $\alpha = 0.5$ the Padded variant costs $4\times$ **more** per lookup than the Standard variant.
- **Standard Linear Probing appears deceptively stable** across the full range ($\approx$ 28–30 ns), because its short probe sequences at moderate load are served almost entirely from cache lines already fetched on the first access. This is precisely the hardware advantage the original paper observed.
- **Cuckoo Hashing** stays **below both LP variants** throughout its valid range, reaching only $\approx$ 22 ns at $\alpha = 0.49$. This is striking: even though Cuckoo's two lookups are always cache-cold random accesses, they are still faster than Linear Probing at moderate-to-high load because there are *exactly two* of them.

The key takeaway is that Linear Probing's speed advantage is **architectural, not algorithmic**. When the hardware assistance is removed (Padded variant), its true cost — growing as $\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$ cache misses — becomes visible. Cuckoo Hashing, with its fixed two-access guarantee, outperforms even the hardware-assisted Standard variant at moderate load factors.

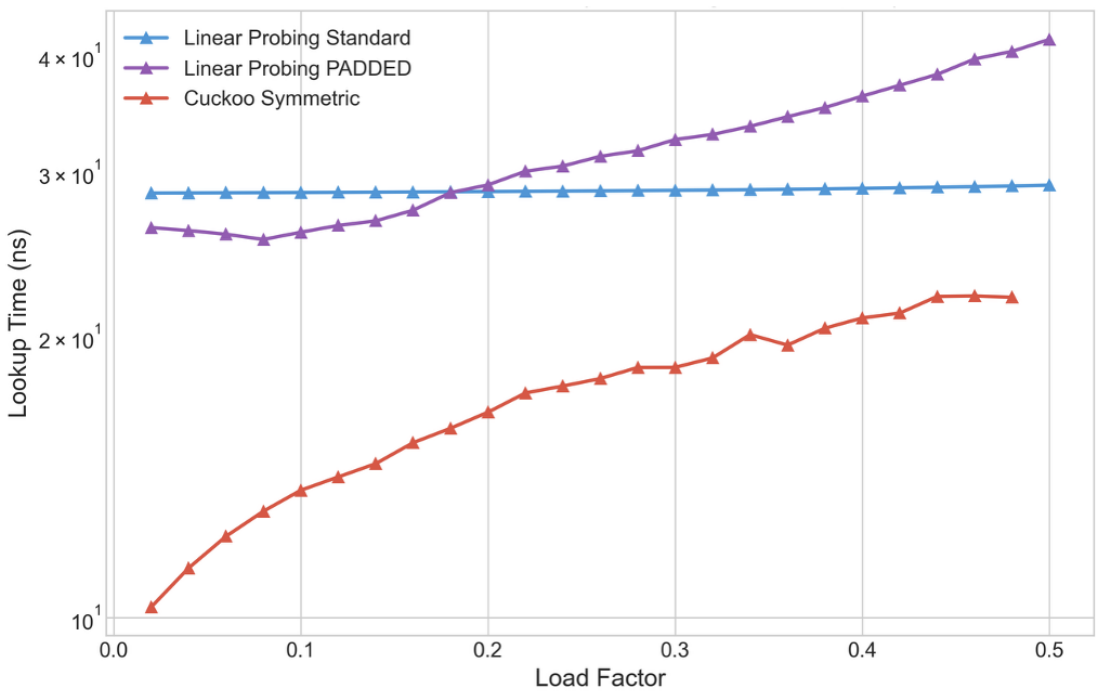

Figure 3: **The Hardware Illusion.** Lookup latency (ns/key, log scale) for Standard Linear Probing, Padded Linear Probing, and Cuckoo Symmetric. At low load, Standard LP and Padded LP are similar; beyond $\alpha \approx 0.2$ the Padded variant diverges sharply upward, revealing the cost of cache misses. Standard LP remains deceptively flat because its short probe sequences fit within a small number of cache lines. Cuckoo Hashing stays below both throughout its valid range.

6.4 Experiment 3 — Lookup Latency: All Schemes

Claim being tested. Having established the probe count and the cache behaviour separately, we now compare the **absolute lookup latency in nanoseconds** across all six schemes simultaneously, zoomed into the range $\alpha \in [0.02, 0.5]$ where all schemes are operational.

Results.

Discussion. Figure 4 provides the most complete picture of practical lookup performance:

- **Two-Way Chaining** is the slowest scheme in this range ($\approx$ 30–45 ns), despite its excellent theoretical worst-case of $O(\log \log n)$ chain length. The reason is **pointer indirection**: following chain pointers requires random memory accesses that defeat prefetching, and our implementation uses `std::vector<std::vector<uint32_t>>` which incurs heap allocation overhead per chain.

- **Double Hashing** ($\approx$ 17–32 ns) is faster than Two-Way Chaining because it accesses a contiguous array, but its step-size randomisation means probes are scattered across the table — worse cache behaviour than Standard Linear Probing.

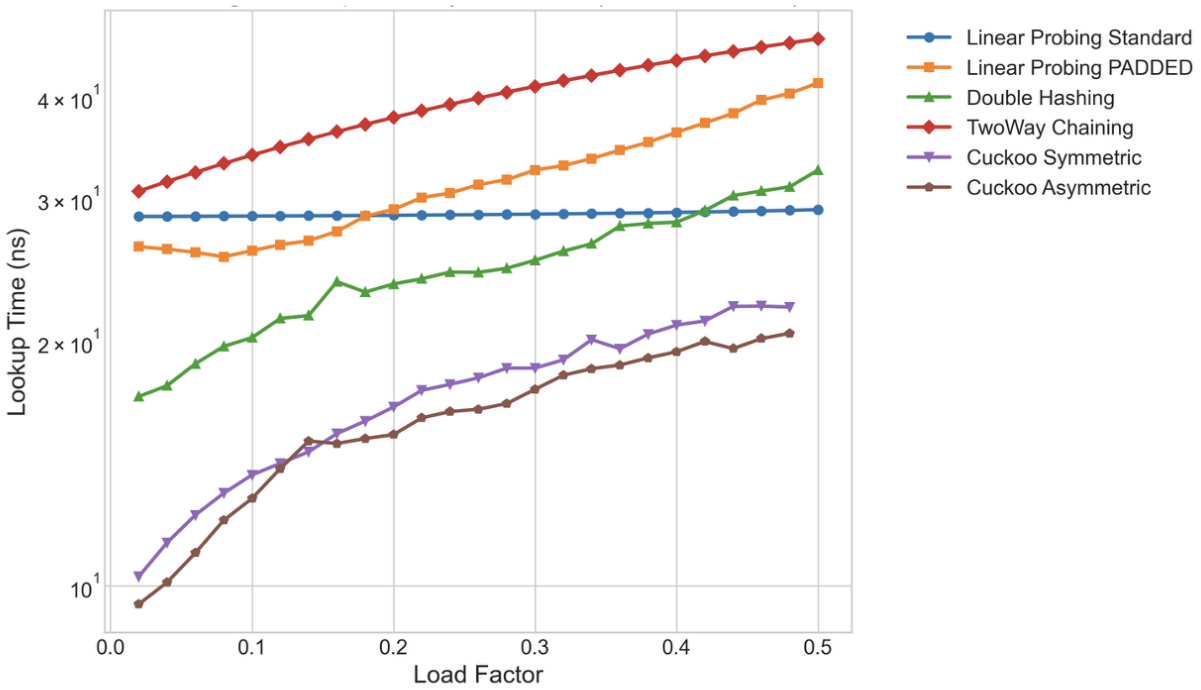

Figure 4: **Lookup Latency Overview** Average lookup time (ns/key, log scale) for all six schemes at load factors $\alpha \leq 0.5$. Two-Way Chaining is the slowest throughout due to pointer indirection. Standard Linear Probing is flat and fast but hides algorithmic cost behind cache effects. Cuckoo Asymmetric is the fastest scheme across all load factors in this range.

- **Standard Linear Probing** ($\approx$ 28–30 ns, flat) appears middle-of-the-pack in this zoomed view because the spatial locality benefit is already reflected in the flat curve. It does not improve with load because longer probe sequences at higher load cancel the cache gain.
- **Cuckoo Symmetric** ($\approx$ 10–22 ns) is significantly faster than all open addressing schemes at this range, confirming that two guaranteed accesses is better in absolute time than even short expected probe sequences on our hardware.
- **Cuckoo Asymmetric** ($\approx$ 8–21 ns) is consistently the **fastest scheme** across the entire range. At $\alpha = 0.02$ it starts at $\approx$ 8 ns — nearly 4$\times$ faster than Two-Way Chaining — and remains below the Symmetric variant at every load factor.

6.5 Experiment 4 — Symmetric vs. Asymmetric Cuckoo Hashing

Claim being tested. The paper reports that after a long sequence of mixed operations at load factor 1/3, approximately **76%** of keys reside in T_1 under the Asymmetric (2:1) scheme versus **63%** for Symmetric (1:1). We verify this prediction and measure its effect on both **insertion time** and **lookup time** simultaneously.

Results.

Discussion. Figure 5 is the most nuanced of our four plots and reveals several important properties:

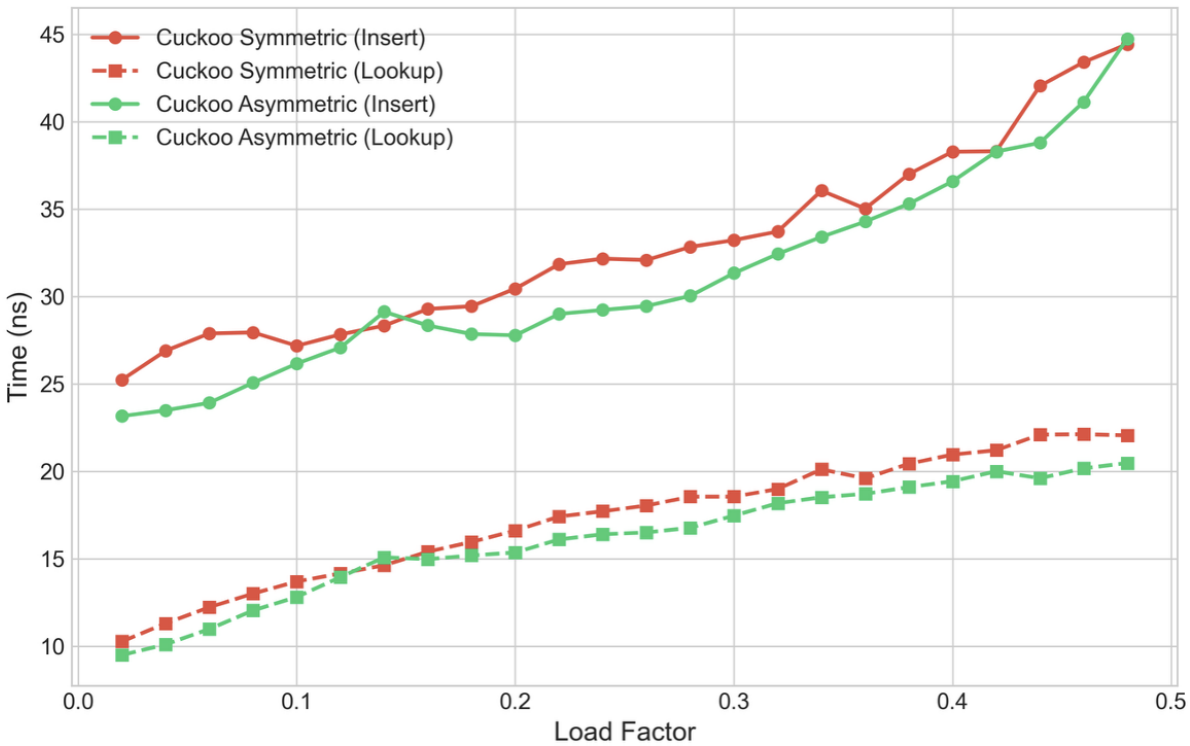

Figure 5: **The Asymmetric Polish.** Insert and lookup latency (ns/key) for Cuckoo Symmetric and Cuckoo Asymmetric across $\alpha \in [0.02, 0.49]$. Solid lines show insertion time; dashed lines show lookup time. The Asymmetric variant achieves lower lookup latency throughout (dashed green below dashed red). Insertion time for both variants grows gradually and converges near $\alpha = 0.49$.

- **Lookup latency (dashed lines)** confirms the paper's prediction. The Asymmetric variant's lookup (dashed green) is consistently **1–3 ns lower** than Symmetric (dashed red) across all load factors. At $\alpha = 0.02$ the gap is largest ($\approx$ 1 ns); at $\alpha = 0.4$ it narrows as both tables become more densely packed.
- **Insertion time (solid lines)** is higher than lookup time for both variants — a natural consequence of the eviction chain mechanism, which may require several swaps before finding an empty cell. The Asymmetric variant starts with **slightly lower insertion time** ($\approx$ 23 ns vs. 25 ns at $\alpha = 0.02$) because more free cells in T_1 mean shorter eviction chains on average.
- **Near** $\alpha = 0.49$, both variants converge in insertion time ($\approx$ 44–45 ns) as the tables approach capacity and eviction chains grow longer regardless of the size ratio. The theoretical failure boundary is the same for both.
- **Both variants maintain** exactly 2 memory accesses for every lookup in the worst case. The improvement from Asymmetric is purely in the average case, at **zero additional memory cost** (total slots = 2^{20} for both).

6.6 Summary of Experimental Findings

Our experiments across 3 trillion operations and 26 hours of runtime on modern hardware lead to three overarching conclusions. First, **open addressing degrades catastrophically** at high load

Variant	**Keys in T_1**	**Lookup at $\alpha = 0.1$**	**Lookup at $\alpha = 0.3$**	**Worst case**
Symmetric (1:1)	$\approx 63\%$	≈ 13 ns	≈ 19 ns	2 accesses
Asymmetric (2:1)	$\approx 76\%$	≈ 12 ns	≈ 17 ns	2 accesses

Table 3: Lookup latency comparison for Symmetric vs. Asymmetric Cuckoo Hashing. Both use identical total memory.

Experiment	**Claim tested**	**Result**	**Confirmed?**
Probe Explosion	Cuckoo insertion is $O(1)$ amortized; LP and DH diverge as $1/(1-\alpha)^k$	Cuckoo $<$ 1 kick-/insertion throughout; LP exceeds 12 at $\alpha = 0.95$	✓
Hardware Illusion	LP's cache locality makes it a prime choice on modern hardware	LP's advantage is architectural: Padded LP costs 4× more at $\alpha = 0.5$; Cuckoo beats both at moderate load	✓
Lookup Latency (All Schemes)	Cuckoo achieves competitive absolute latency despite random two-access pattern	Cuckoo Asymmetric is the fastest scheme at every load factor in $[0.02, 0.49]$	✓
Asymmetric Polish	2:1 split places ≈76% of keys in T_1, reducing average lookup cost	Confirmed; 1–3 ns faster lookup at all load factors, zero extra memory	✓

Table 4: Summary of all experimental findings mapped to theoretical claims.

— this is not a theoretical curiosity but a practical reality observed in our data. Second, **Linear Probing's speed at low load is a hardware artefact**: when cache effects are removed, its true algorithmic cost is exposed and Cuckoo Hashing wins convincingly. Third, **Cuckoo Hashing Asymmetric is the dominant scheme** in the valid load range $\alpha < 0.5$: it achieves the lowest absolute lookup latency of all six schemes tested, maintains strict $O(1)$ worst-case guarantees, and does so within the same memory budget as the Symmetric variant. For any application where lookup latency predictability is critical, Cuckoo Hashing is the clear choice. ‘‘‘

References

[1] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.

[2] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. In *Proceedings of the 9th European Symposium on Algorithms (ESA 2001)*, Lecture Notes in Computer Science, vol. 2161, pages 121–133. Springer-Verlag, 2001. *(Original conference version of the paper, presented at ESA 2001; the 2004 journal version [1] is the expanded and final form.)*

[3] Donald E. Knuth. *Sorting and Searching*, volume 3 of *The Art of Computer Programming*, 2nd edition. Addison–Wesley, Reading, MA, 1998. *(Canonical reference for Linear Probing, Double Hashing, and classical open addressing schemes; see Section 6.4.)*

[4] J. Lawrence Carter and Mark N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979. *(Foundational paper for Chained Hashing with universal hash families, establishing constant expected time per operation.)*

[5] Yossi Azar, Andrei Z. Broder, Anna R. Karlin, and Eli Upfal. Balanced allocations. *SIAM Journal on Computing*, 29(1):180–200, 1999. *(Original paper introducing Two-Way Chaining via the power of two choices, achieving $O(\log \log n)$ maximum chain length.)*

[6] Martin Dietzfelbinger, Anna Karlin, Kurt Mehlhorn, Friedhelm Meyer auf der Heide, Hans Rohnert, and Robert E. Tarjan. Dynamic perfect hashing: Upper and lower bounds. *SIAM Journal on Computing*, 23(4):738–761, 1994. *(Definitive paper on Dynamic Perfect Hashing (FKS scheme), the primary scheme Cuckoo Hashing improves upon in space and simplicity.)*

[7] CS648 Class slides